in the chain have a lower hierarchy value than the prior one, which may be impractical in some cases.

This example also demonstrates another point, the use of the `std::lock_guard<>` template with a user-defined mutex type. `hierarchical_mutex` is not part of the standard but is easy to write; a simple implementation is shown in listing 3.8. Even though it's a user-defined type, it can be used with `std::lock_guard<>` because it implements the three member functions required to satisfy the mutex concept: `lock()`, `unlock()`, and `try_lock()`. You haven't yet seen `try_lock()` used directly, but it's fairly simple: if the lock on the mutex is held by another thread, it returns `false` rather than waiting until the calling thread can acquire the lock on the mutex. It may also be used by `std::lock()` internally, as part of the deadlock-avoidance algorithm.

Listing 3.8. A simple hierarchical mutex

```
class hierarchical_mutex
{
    std::mutex internal_mutex;
    unsigned long const hierarchy_value;
    unsigned long previous_hierarchy_value;
    static thread_local unsigned long this_thread_hierarchy_value;     <-(1)

    void check_for_hierarchy_violation()
    {
        if(this_thread_hierarchy_value <= hierarchy_value)     <-(2)
        {
            throw std::logic_error("mutex hierarchy violated");
        }
    }
    void update_hierarchy_value()
    {
        previous_hierarchy_value=this_thread_hierarchy_value;     <-(3)
        this_thread_hierarchy_value=hierarchy_value;
    }
public:
    explicit hierarchical_mutex(unsigned long value):
        hierarchy_value(value),
        previous_hierarchy_value(0)
```